

Documentación de la aplicación

Escáner de Personalidad Pokémon

Versión ampliada de la documentación. Esta edición incorpora el funcionamiento real de la generación de los perfiles Pokémon a partir del cuaderno Test_Pokemon(1).ipynb, además de la aplicación final app.py.

Componente	Tecnología / método
Aplicación web	Python + Streamlit
Cuestionario	50 preguntas × 4 respuestas
Perfil del usuario	Vector de 5 dimensiones
Perfil Pokémon	TF-IDF sobre descripciones de PokéAPI
Normalización Pokémon	Escala 0-10 por dimensión
Comparación	Similitud del coseno
Base de referencia	1.025 especies descargadas
Gimnasio	Recuento de tipos asociados a las respuestas
Imagen final	PokéAPI

Fuente principal: código de app.py y notebook Test_Pokemon(1).ipynb. El notebook muestra la construcción de la base de perfiles mediante extracción de descripciones, TF-IDF, palabras clave, suma por dimensión y normalización a 0-10. [filecite1file0L318-L376](#)

1. Visión general

La aplicación tiene dos resultados relacionados pero calculados de forma diferente. El primero es el Pokémon que más se parece al perfil del usuario. El segundo es el tipo de gimnasio que lideraría.

La clave para entender el proyecto completo es que los perfiles Pokémon no están escritos manualmente en app.py. Se generan previamente en el notebook a partir de las descripciones textuales de los Pokémon. Después se exportan a vectores_pokemon.json, y app.py utiliza ese archivo como base de comparación. El propio notebook exporta las columnas id, nombre y las cinco dimensiones, descartando las descripciones largas para reducir el tamaño de la información que necesita la aplicación web.

filecite[turn1file0]L593-L606

Por tanto, el sistema completo puede entenderse como una cadena de dos fases: preprocesamiento de la Pokédex y ejecución del test.

2. Fase 1: construcción de los perfiles Pokémon

2.1. Obtención de las descripciones

El notebook comienza conectándose a PokéAPI y recorriendo los identificadores desde 1 hasta 1025. Para cada especie solicita el endpoint pokemon-species/{id}. Si la respuesta es correcta, obtiene el nombre y todas las entradas de descripción cuya lengua es inglés.

```
for i in range(1, limite + 1):
    url = f"https://pokeapi.co/api/v2/pokemon-species/{i}/"
    respuesta = requests.get(url)
    if respuesta.status_code == 200:
        datos = respuesta.json()
```

Las distintas descripciones inglesas se unen en un único texto. Además, se sustituyen saltos de línea, saltos de página y ciertos caracteres para dejar un texto continuo. El notebook ejecutado informa de que se descargaron 1.025 Pokémon. filecite[turn1file0]L91-L133

2.2. Por qué se utiliza texto

La idea es evitar asignar manualmente una personalidad a cada Pokémon. En lugar de decir directamente que un Pokémon es 'intelectual = 8', el sistema intenta obtener esa información de cómo se describe al Pokémon en la Pokédex.

Por ejemplo, si una descripción utiliza repetidamente conceptos relacionados con dormir, calma o amabilidad, esas palabras pueden contribuir a la dimensión Calmado/Afable. Si aparecen conceptos relacionados con inteligencia, mente o cálculo, pueden contribuir a Intelectual/Astuto.

3. TF-IDF: cómo se transforma una descripción en números

3.1. El problema de contar palabras

Contar simplemente cuántas veces aparece una palabra no sería suficiente. Una palabra muy común en casi todas las descripciones aporta poca información para distinguir un Pokémon de otro. En cambio, una palabra que aparece mucho en un Pokémon y muy poco en el resto puede ser muy característica.

El notebook utiliza TF-IDF (Term Frequency-Inverse Document Frequency) precisamente para ponderar esa importancia. El propio notebook explica la idea mediante el ejemplo de la palabra 'sleep': si aparece mucho en Snorlax y poco en el resto, su peso puede ser alto para Snorlax. filecite[turn1file0]L318-L320

3.2. Las cinco dimensiones

Dimensión	Palabras clave definidas en el notebook
-----------	---

Agresivo / Audaz	angry, rage, fierce, attack, fight, violent, cruel, destroy, territorial
Calmado / Afable	calm, gentle, peace, sleep, quiet, friendly, heal, soft, kind
Enérgico / Activo	fast, speed, run, quick, energy, active, jump, electric, agile
Intelectual / Astuto	smart, intelligent, brain, mind, wisdom, clever, genius, calculate
Misterioso / Oscuro	dark, shadow, trick, curse, ghost, night, creepy, hide, sinister

Estas palabras están definidas explícitamente en el diccionario `diccionario_personalidad` del notebook. Después se juntan en un único vocabulario y se crea un `TfidfVectorizer` restringido a ese vocabulario y con stop words en inglés. `filecite\turn1file0\L338-L357`

3.3. Qué produce TF-IDF

El resultado es una matriz numérica: cada fila corresponde a un Pokémon y cada columna a una palabra clave. Una celda representa el peso TF-IDF de esa palabra en la descripción del Pokémon.

```
matriz_tfidf = vectorizador.fit_transform(df_pokemon['descripcion'])
```

La matriz se convierte después en un `DataFrame` para poder seleccionar las columnas correspondientes a cada dimensión. `filecite\turn1file0\L353-L360`

4. De palabras clave a un perfil de personalidad Pokémon

4.1. Suma de los pesos por dimensión

El paso fundamental es agrupar las palabras. Para cada dimensión se toman sus palabras clave y se suman sus pesos TF-IDF.

```
for dimension, palabras in diccionario_personalidad.items():
    df_pokemon[dimension] = df_vectores[palabras].sum(axis=1)
```

Así, un Pokémon deja de estar representado por cientos de palabras y pasa a estar representado por cinco números:

```
[Agresivo/Audaz, Calmado/Afable, Energico/Activo, Intelectual/Astuto, Misterioso/Oscuro]
```

Estos cinco valores son el vector de personalidad del Pokémon. El proceso completo está implementado en el notebook entre la creación de la matriz TF-IDF y la suma de las palabras por dimensión. `filecite\turn1file0\L361-L365`

4.2. Normalización a una escala 0-10

Los valores obtenidos todavía no están en una escala cómoda. Por eso el notebook normaliza cada dimensión por separado.

```
max_val = df_pokemon[dimension].max()
df_pokemon[dimension] = round(
    (df_pokemon[dimension] / max_val) * 10, 2
)
```

El Pokémon que tenga el valor máximo de una dimensión recibe 10 en esa dimensión; los demás reciben una puntuación proporcional. Si una dimensión no tiene ninguna coincidencia, se establece a 0.

`filecite\turn1file0\L366-L373`

Es importante entender que 10 no significa '10 puntos psicológicos absolutos'. Significa que ese Pokémon alcanza el máximo de esa dimensión dentro del conjunto utilizado para normalizarla.

4.3. Ejemplos reales del notebook

Pokémon	Agresivo	Calmado	Enérgico	Intelectual	Misterioso
---------	----------	---------	----------	-------------	------------

Primeape	5,31	0,68	2,47	2,80	0,00
Alakazam	0,00	0,00	0,00	8,43	0,00
Snorlax	0,00	8,85	0,00	1,45	0,00

El notebook muestra estos valores como comprobación. Alakazam destaca en Intelectual/Astuto y Snorlax en Calmado/Afable, mientras que Primeape presenta una combinación más repartida.

[filecite\turn1file0\L375-L400](#)

5. El archivo vectores_pokemon.json

Una vez calculados los perfiles, el notebook crea un DataFrame final con únicamente id, nombre y las cinco dimensiones. La descripción completa se elimina de la versión destinada a la web.

```
columnas_exportar = ['id', 'nombre'] + columnas_dimensiones
df_final = df_pokemon[columnas_exportar]
df_final.to_json('vectores_pokemon.json',
orient='records', force_ascii=False)
```

Esto es una decisión importante de arquitectura. La aplicación no necesita volver a ejecutar TF-IDF cada vez que un usuario hace el test. El cálculo pesado se realiza una vez en la fase de preparación y la aplicación utiliza el resultado ya procesado. [filecite\turn1file0\L593-L606](#)

6. Fase 2: cómo se construye el perfil del usuario

6.1. Cada respuesta tiene dos efectos

En app.py cada una de las cuatro opciones de cada pregunta contiene dos piezas de información:

- perfil: un vector de cinco números que representa cuánto aporta la respuesta a cada dimensión.
- tipos: una lista de tres tipos Pokémon que recibe un voto si se selecciona esa respuesta.

Por ejemplo, la primera pregunta contiene una opción con [4,0,3,0,0] y los tipos Fuego, Lucha y Roca. Otra contiene [0,5,0,0,0] y los tipos Agua, Hada y Planta. [filecite\turn5file0\L31-L42](#)

6.2. Suma de las 50 respuestas

El perfil comienza como un vector de cinco ceros. Para cada respuesta se recupera su vector y se suma al acumulador.

```
perfil_personalidad = np.zeros(5)
perfil_personalidad += np.array(datos["perfil"])
```

Después de las 50 preguntas, el vector contiene la suma de todas las aportaciones. No se calcula una media: se acumulan las puntuaciones. El código realiza exactamente esta operación para cada respuesta. [filecite\turn5file0\L321-L331](#)

6.3. Ejemplo conceptual

Si un usuario tuviera solamente tres respuestas hipotéticas:

```
Respuesta 1 -> [4, 0, 3, 0, 0]
Respuesta 2 -> [0, 5, 0, 0, 0]
Respuesta 3 -> [0, 0, 0, 5, 2]

Total -> [4, 5, 3, 5, 2]
```

Con 50 preguntas ocurre exactamente lo mismo, pero con 50 vectores sumados. El orden de las dimensiones se mantiene siempre igual: Agresivo/Audaz, Calmado/Afable, Energico/Activo, Intelectual/Astuto y Misterioso/Oscuro. [filecite\turn5file0\L27-L28](#)

7. Cómo se compara el usuario con los Pokémon

7.1. Matriz Pokémon

Cuando se inicia la aplicación, vectores_pokemon.json se carga en un DataFrame mediante Pandas. app.py selecciona las cinco columnas de dimensiones.

```
df_pokemon = pd.read_json("vectores_pokemon.json")

columnas_dimensiones = [
    'Agresivo_Audaz',
    'Calmado_Afable',
    'Energico_Activo',
    'Intelectual_Astuto',
    'Misterioso_Oscuro'
]
```

Esto genera conceptualmente una matriz donde cada fila es un Pokémon y cada columna una dimensión.

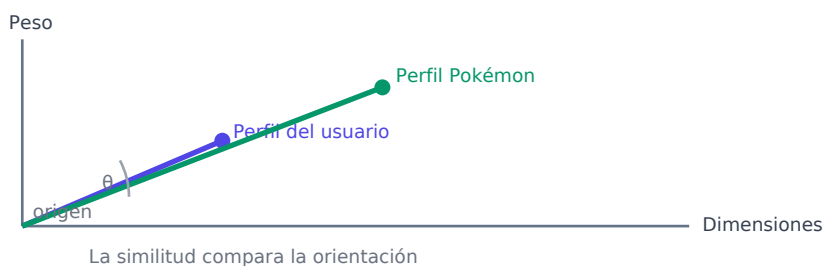
```
Matriz Pokémon =
[
    [A1, C1, E1, I1, M1],
    [A2, C2, E2, I2, M2],
    ...
    [A1025, C1025, E1025, I1025, M1025]
]
```

La aplicación carga el JSON con caché de Streamlit, de modo que el fichero no tiene que volver a leerse innecesariamente durante la interacción. [filecite\turn5file0\L21-L28](#)

7.2. Similitud del coseno

El algoritmo utiliza cosine_similarity. En lugar de comparar cada número individualmente, compara la orientación de los vectores. Matemáticamente:

$$\cos(\theta) = (A \cdot B) / (||A|| \cdot ||B||)$$



Un valor cercano a 1 significa que ambos perfiles apuntan en direcciones muy parecidas. Esto permite que el algoritmo valore la proporción entre las dimensiones, no simplemente que los números sean pequeños o grandes.

Por ejemplo, un usuario con una proporción aproximada de 80 % Intelectual y 20 % Misterioso debería parecerse a un Pokémon con una distribución semejante aunque las magnitudes absolutas sean diferentes. El notebook explica expresamente esta propiedad de la similitud del coseno.

[filecite\turn1file0\L556-L558](#)

7.3. Selección del ganador

```
vector_usuario = perfil_personalidad.reshape(1, -1)
coincidencias = cosine_similarity(
    vector_usuario,
    df_pokemon[columnas_dimensiones].values
)[0]

mejor_indice = np.argmax(coincidencias)
mejor_pokemon = df_pokemon.iloc[mejor_indice]
```

La función produce una coincidencia para cada fila de la base de Pokémon. np.argmax obtiene la posición del valor máximo, y esa fila se convierte en el Pokémon ganador. La aplicación transforma después el valor en porcentaje y lo muestra con un decimal. [filecite\turn5file0\L333-L339](#)

8. Ejemplo completo de comparación

El notebook incluye una prueba antes de exportar los vectores. Simula un usuario con:

```
puntuacion_usuario = [1.0, 9.5, 2.0, 7.0, 0.5]
```

Ese perfil representa un usuario muy calmado, bastante intelectual, poco activo y prácticamente nada misterioso/agresivo. Se compara simultáneamente contra todos los Pokémon.

El resultado de prueba muestra:

Posición	Pokémon	Coincidencia
1	Abra	0,971014
2	Slowking	0,968992
3	Herdier	0,960600

La prueba obtiene Abra como primera coincidencia con 0,971014, seguido de Slowking y Herdier. Este ejemplo demuestra que el sistema no selecciona un Pokémon mediante una regla fija de tipo, sino comparando vectores. [filecite\turn1file0\L574-L634](#)

9. El gimnasio se calcula de otra manera

Es fundamental no confundir el perfil Pokémon con el tipo de gimnasio. El Pokémon se obtiene mediante similitud vectorial. El gimnasio se obtiene mediante votos.

Cada respuesta tiene exactamente tres tipos asociados. Cuando el usuario selecciona una respuesta, se suma 1 a cada uno de esos tres tipos.

```
for tipo in datos["tipos"]:  
    puntuacion_tipos[tipo] += 1
```

Al finalizar, se dispone de un contador para los 18 tipos. El tipo con mayor número de votos se selecciona con max(...). [filecite\turn5file0\L325-L339](#)

Por tanto, una respuesta puede influir simultáneamente en dos sistemas: aporta una cantidad al vector de personalidad y aporta tres votos de tipo.

10. Ejemplo de una respuesta completa

Consideremos una opción real del test:

```
A) Tomo el mando, decido la hora y compro los billetes del tirón  
  
perfil = [4, 0, 3, 0, 0]  
tipos = [Dragón, Fuego, Lucha]
```

Esta respuesta hace dos cosas:

- Añade 4 puntos a Agresivo/Audaz.
- Añade 3 puntos a Enérgico/Activo.
- Añade un voto a Dragón.
- Añade un voto a Fuego.
- Añade un voto a Lucha.

La respuesta no 'elige' directamente un Pokémon. Solo modifica el vector del usuario y el contador de tipos. El Pokémon se decide posteriormente, cuando se ha completado todo el cuestionario. La estructura de esta respuesta está presente en app.py. [filecite turn5file0 L43-L47](#)

11. Validación y flujo de la aplicación web

11.1. Formulario

Streamlit utiliza st.form para agrupar las 50 preguntas. Cada una se representa mediante un st.radio. El parámetro index=None hace que las opciones comiencen sin seleccionar.

El cálculo no se ejecuta hasta pulsar el botón Analizar mi Personalidad. [filecite turn5file0 L300-L312](#)

11.2. Validación

Antes de calcular nada, la aplicación comprueba qué respuestas siguen siendo None. Si existe alguna, muestra al usuario las preguntas que faltan y no continúa con el análisis. [filecite turn5file0 L314-L320](#)

11.3. Resultado

Cuando todo está contestado, la aplicación calcula el vector, obtiene el Pokémon, calcula el gimnasio, consulta PokéAPI y prepara el informe final. [filecite turn5file0 L321-L357](#)

12. Consulta final a PokéAPI

El perfil Pokémon ya está almacenado en el JSON, pero la imagen no se guarda allí. Una vez elegido el Pokémon, app.py utiliza su identificador para consultar PokéAPI.

```
url_api = f"https://pokeapi.co/api/v2/pokemon/{mejor_pokemon['id']}/"
respuesta_api = requests.get(url_api).json()

sprites = respuesta_api.get('sprites', {}).get('other', {}).get('official-artwork', {})

imagen_url = sprites.get('front_default')
```

La aplicación utiliza accesos mediante get() para evitar que una ausencia de la imagen provoque un acceso directo a una clave inexistente. Si no obtiene imagen, muestra un aviso al usuario.

[filecite turn5file0 L341-L367](#)

Esta llamada es únicamente para enriquecer visualmente el resultado. La decisión matemática del Pokémon ya se ha realizado antes de consultar la imagen.

13. Cálculo de los porcentajes del perfil del usuario

Una vez terminado el test, la aplicación convierte las puntuaciones acumuladas del usuario en porcentajes relativos.

```
total_puntos = np.sum(perfil_personalidad)
porcentajes = (perfil_personalidad / total_puntos) * 100
```

Si, por ejemplo, el vector fuese [20, 40, 10, 25, 5], el total sería 100 y los porcentajes serían exactamente 20 %, 40 %, 10 %, 25 % y 5 %. El programa ordena después las cinco dimensiones de mayor a menor para localizar el rasgo principal y el secundario. [filecite turn5file0 L349-L355](#)

El segundo rasgo solo se incluye en el texto final si supera el 15 %. La aplicación muestra el nombre del Pokémon, la afinidad, el gimnasio y el análisis de personalidad. [filecite turn5file0 L357-L384](#)

14. Arquitectura completa del proyecto

La arquitectura se puede dividir en dos momentos claramente separados.

14.1. Preparación de datos

- Se descargan 1.025 especies desde PokéAPI.
- Se recopilan las descripciones inglesas.
- Se limpian y concatenan los textos.
- Se aplica TF-IDF sobre las palabras clave.
- Se suman las palabras de cada una de las cinco dimensiones.
- Cada dimensión se normaliza a una escala 0-10.
- Se exportan los vectores finales a vectores_pokemon.json.

Este proceso pertenece al notebook y no se ejecuta cada vez que un usuario hace el test. [filecite\turn1file0\L91-L133](#) [filecite\turn1file0\L338-L373](#)

14.2. Ejecución de la aplicación

- Se carga vectores_pokemon.json.
- Se muestran las 50 preguntas.
- Se valida que todas estén contestadas.
- Se suman los vectores de las respuestas.
- Se cuentan los votos de tipos.
- Se calcula la similitud del coseno contra todos los Pokémon.
- Se selecciona el máximo.
- Se obtiene la imagen mediante PokéAPI.
- Se presenta el resultado.

Este segundo proceso está implementado en app.py. [filecite\turn5file0\L21-L28](#) [filecite\turn5file0\L300-L384](#)

15. Tecnologías y responsabilidad de cada una

Tecnología	Uso
Python	Lenguaje en el que se implementan tanto el notebook como la aplicación.
Pandas	Tablas de datos, lectura del JSON y preparación de la matriz Pokémon.
NumPy	Vectores, acumulación de puntuaciones y operaciones numéricas.
scikit-learn	TF-IDF y similitud del coseno.
Requests	Comunicación HTTP con PokéAPI.
Streamlit	Interfaz web de la versión final.
PokéAPI	Fuente de descripciones durante la preparación y fuente de imagen durante el test.
JSON	Formato intermedio que contiene los perfiles finales utilizados por la aplicación.

16. Qué significa realmente el resultado

La aplicación utiliza la palabra 'personalidad', pero técnicamente el resultado debe interpretarse como una clasificación temática construida mediante reglas y características textuales. No se trata de un

diagnóstico psicológico ni de un modelo clínico.

El perfil Pokémon procede de descripciones de Pokédex y de un diccionario de palabras diseñado para este proyecto. El perfil del usuario procede de reglas manuales asignadas a las respuestas. La similitud del coseno únicamente determina qué vector de referencia tiene una orientación más parecida.

Esto no resta valor al objetivo del proyecto: para una aplicación lúdica, el método permite explicar de manera transparente por qué un resultado está relacionado con otro. Pero es importante documentarlo correctamente y no presentar el algoritmo como una herramienta psicológica validada.

17. Ventajas del diseño actual

- Separación entre preparación y ejecución: TF-IDF no se recalcula para cada usuario.
- Explicabilidad: cada respuesta tiene una contribución explícita.
- Base homogénea: todos los Pokémon se convierten al mismo espacio de cinco dimensiones.
- Comparación escalable: añadir Pokémon a la base no obliga a modificar el algoritmo del test.
- Interfaz sencilla: Streamlit permite construir el formulario directamente desde Python.
- Resultado enriquecido: el JSON resuelve la parte matemática y PokéAPI aporta la imagen.

El notebook demuestra además que el archivo final se reduce a las columnas necesarias para la web, evitando transportar las descripciones completas. `filecite[turn1file0]L593-L603`

18. Limitaciones técnicas actuales

18.1. Calidad del perfil Pokémon

El perfil depende de las palabras clave seleccionadas. Si una descripción expresa una idea de forma diferente y no utiliza ninguna de las palabras incluidas, esa señal no será detectada.

18.2. Dependencia del idioma

El proceso de extracción filtra explícitamente las descripciones inglesas. Por tanto, las palabras clave también están definidas en inglés. `filecite[turn1file0]L108-L116`

18.3. Normalización relativa

Cada dimensión se normaliza respecto al máximo de esa misma dimensión. Por ello, un 8 en Intelectual y un 8 en Calmado no implican necesariamente la misma cantidad absoluta de evidencia textual; representan posiciones relativas dentro de dimensiones distintas.

18.4. TF-IDF no entiende el significado de una frase

El sistema trabaja con pesos de palabras. No interpreta de manera semántica completa una oración. Por ejemplo, una palabra relacionada con 'attack' puede aportar a Agresivo/Audaz aunque el contexto lingüístico pueda ser más complejo.

18.5. El usuario y los Pokémon se generan de manera diferente

El vector Pokémon procede de TF-IDF + palabras clave + normalización. El vector del usuario procede de reglas manuales en las respuestas. Ambos terminan en el mismo espacio de cinco dimensiones, y esa compatibilidad es lo que permite compararlos mediante coseno.

18.6. Robustez de la API

app.py comprueba si la imagen existe mediante accesos seguros, pero la petición HTTP no incluye un timeout ni un manejo explícito de excepciones de red. `filecite[turn5file0]L341-L347`

19. Mejoras recomendadas

Prioridad	Mejora	Motivo
Alta	Separar el motor matemático de Streamlit	Permitir pruebas independientes y reutilización.
Alta	Controlar errores y timeout de PokéAPI	Evitar que un fallo de red rompa el flujo.
Alta	Validar el esquema del JSON	Detectar columnas o datos incorrectos antes del cálculo.
Media	Guardar el diccionario de palabras en un archivo	Facilitar ajustar las dimensiones sin editar código.
Media	Versionar la generación de vectores	Saber con qué corpus y palabras se creó cada JSON.
Media	Añadir pruebas unitarias	Comprobar suma de perfiles, selección y empates.
Media	Mostrar un Top 3	Explicar mejor la afinidad y evitar que el resultado parezca arbitrario.

20. Explicación resumida para presentar el proyecto

Si hubiera que explicar el funcionamiento en una presentación oral, puede resumirse así:

Primero, se descargan las descripciones de 1.025 Pokémon desde PokéAPI. Segundo, se utiliza TF-IDF para medir qué palabras son características de cada descripción. Tercero, esas palabras se agrupan en cinco dimensiones de personalidad y se obtiene un vector de cinco números por Pokémon. Cuarto, cada dimensión se normaliza a una escala de 0 a 10 y todos esos vectores se guardan en un JSON.

Después, en la aplicación, el usuario responde 50 preguntas. Cada respuesta añade puntos a un vector de cinco dimensiones y tres votos a tipos Pokémon. Una vez terminado el test, el vector del usuario se compara mediante similitud del coseno con los 1.025 vectores Pokémon. El de mayor similitud es el resultado. Paralelamente, el tipo con más votos determina el gimnasio.

Finalmente, la aplicación consulta PokéAPI para mostrar la ilustración del Pokémon ganador y presenta la afinidad, el gimnasio y los rasgos principales del usuario.

21. Referencia de código

Las partes fundamentales de la aplicación final se encuentran organizadas de la siguiente manera:

Código	Responsabilidad
app.py, líneas 9-28	Imports, configuración, carga del JSON y dimensiones.
app.py, líneas 31-298	Definición de las 50 preguntas y reglas.
app.py, líneas 300-312	Construcción del formulario Streamlit.
app.py, líneas 314-339	Validación, suma del perfil, votos, coseno y ganador.
app.py, líneas 341-347	Consulta de PokéAPI para la imagen.
app.py, líneas 349-384	Porcentajes y presentación del resultado.
Notebook, líneas 91-133	Extracción de 1.025 especies y descripciones.
Notebook, líneas 338-373	TF-IDF, dimensiones y normalización.
Notebook, líneas 574-606	Prueba del coseno y exportación del JSON.

22. Conclusión

La parte más importante del proyecto es la conexión entre dos mundos: texto y vectores. Las descripciones de los Pokémon se convierten primero en información numérica mediante TF-IDF. Esa información se reduce a cinco dimensiones que forman el perfil de cada Pokémon. El usuario también termina representado por cinco números, aunque esos números proceden de las respuestas del cuestionario.

Una vez que ambos están en el mismo espacio vectorial, la similitud del coseno permite compararlos de forma matemática. El resultado no es una elección arbitraria ni una tabla que diga directamente 'respuesta X = Pokémon Y': es la consecuencia de acumular las respuestas del usuario y buscar el vector Pokémon cuya orientación sea más parecida.

En paralelo, el sistema de gimnasio funciona como un contador de afinidades de tipo. Esta separación hace que el resultado final combine una afinidad de personalidad con una especialidad de tipo, manteniendo cada cálculo sencillo y explicable.